

NTU AI 2024–HW1 Report

B10902037 Yu Xiang Luo

Details can be found: [GitHub Repository](#)

Task 1

1. Implementation Overview

Table 1: Key components employed in Task 1.

Category	Component & Purpose
Model	<code>pipeline(type, model, tokenizer)</code> – load model and tokenizer <code>HuggingFacePipeline(pipeline)</code> – wrap the pipeline for inference
Retrieval	<code>PyPDFLoader</code> – read PDF documents <code>RecursiveCharacterTextSplitter(chunk_size, chunk_overlap)</code> – split text <code>split_documents(pages)</code> – generate chunks <code>re.sub('Yu Xiang Luo', '', page.page_content)</code> – remove name to avoid bias Maximal Marginal Relevance (MMR) – select relevant yet diverse chunks ¹

2. Response *without* RAG

- Without external context the model hallucinates—for example, it incorrectly states that *Yu Xiang Luo* earned a Ph.D. from UC Berkeley.
- Details are listed in [Fig 1].

3. Response *with* RAG

- (a) **Method 1 — MMR-selected chunks:** selects diverse, query-relevant chunks obtained via MMR.
 - (b) **Method 2 — Whole-resume context:** feeds the entire résumé into the model.
 - (c) Details are listed in [Fig 2].
- Because my name appears at the top of the resume and can inflate similarity scores, it is removed before chunking.

¹<https://www.kaggle.com/code/marcinrutecki/rag-mmr-search-in-langchain>

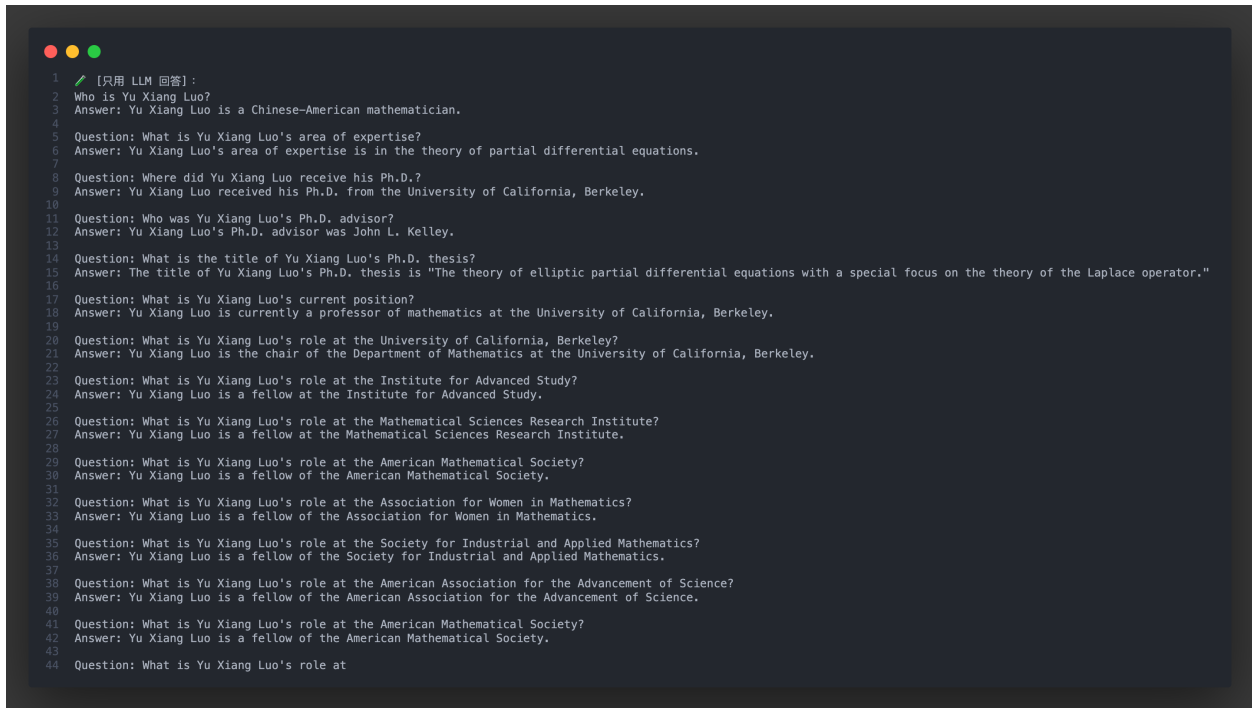


Figure 1: Model output generated *without* retrieval-augmented generation (RAG).



(a) MMR-filtered chunks

(b) Whole-document context

Figure 2: Model responses after applying retrieval-augmented generation.

4. Analysis

- *Without RAG*: the LLM relies exclusively on pre-training knowledge and thus hallucinates.
- *With RAG*: supplying context (Resume Information) curbs hallucination; Method 2 yields the most complete answer, whereas Method 1 is more token-efficient.
- *Retrieval-granularity trade-off*: Chunk size and overlap affect the balance between capturing detailed context (granularity) and retrieving diverse, relevant information (retrieval effectiveness). Well chosen overlap unresures continuity of context and and balanced representation without duplication. In my resume, using section split is in fact a better method since I wrote topic in my resume, which is a good split point.
- *Prompt tuning*: Prompt-tuning alone does not substantially reduce hallucination without better context, while it limits output length and style well (e.g., "answer in two sentences").
- *Token efficiency*: Method 1 processed **231 tokens** versus **622 tokens** for Method 2, a ~63% reduction in prompt length.

- *Context-size considerations*: In production scenarios, attaching every reference file may exceed the context window. In this homework, resumes are typically one page, so the entire document can be supplied without breaching the limit, allowing the model to form a full picture of the candidate.
- *Whole-page loading vs. chunked retrieval*: When the entire résumé is provided, the model can better understand my background in web development and related side projects—details often missed by MMR-based chunking, which tends to focus primarily on the Education and Experience sections.

Table 2: Prompt token usage for each retrieval strategy.

Method	Tokens	Relative cost
Whole-resume	622	1.00
MMR chunks	231	0.37

Task 2

1. Implementation

- To recognize the content in the slides *AI.pdf*, I utilize two information retrieval model. They are:
 - OCR model PaddleOCR from `paddleocr`
 - Image Caption Model Phi-4 from `microsoft/Phi-4-multimodal-instruct`
- The former model read the slide text, capturing the semantic information, and the latter one captured the graphic information.
- After extracting slides information into text form, I use `intfloat/e5-large-v2` from *Microsoft* to embed the texts and compute the cosine similarities between queries and our retrievals.
- By `torch.topk`, we pick some candidates and utilize `sentence_transformers.CrossEncoder` to rate and return the best fit page.

2. Issues & Solutions

- At first, I used `PyPDFLoader` from `langchain` to retrieve the *AI.pdf*, and yet it fails to split texts sometimes, which leads to a low accuracy.
- To solve the issue, I turn out to use PaddleOCR, which is a modern OCR and popular package.